Ders 6 Genişletilmiş Notu: Öğrenmenin Transferi, Çok Görevli Eğitim, Hiperparametre Optimizasyonu, Aktif Örnekleme ve Anlamsal Uzay Ödev Sunumları

Ders 6 Genişletilmiş Notu: Öğrenmenin Transferi, Çok Görevli Eğitim, Hiperparametre Optimizasyonu, Aktif Örnekleme ve Anlamsal Uzay Ödev Sunumları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bölüm 4'ün Amacı ve Önceki Dersin Tekrarı
- 2. Öğrenmenin Transferi
- 3. Çok Görevli Eğitim
- 4. Birbiriyle İlgisiz Veri Kümelerini Birlikte Eğitmek
- 5. Çok Görevli Eğitim ve Çok Kipli Modeller
- 6. Hiperparametre Optimizasyonu
- 7. Aktif Örnekleme
- 8. Ödev Sunumları: Metin Sınıflandırmada Anlamsal Uzayların Karşılaştırılması
- 9. Sunumlarda Hocanın Geri Bildirimleri
- Örnek Sorular
- Dersin Toparlaması

1. Bölüm 4'ün Amacı ve Önceki Dersin Tekrarı

Dersin ilk yarısında hoca Bölüm 4'e devam etti, ikinci yarısı öğrencilerin ödev sunumlarına ayrıldı. Bölüm 4'ün içeriği "makine öğrenmesi versiyon 2" ya da makine öğrenmesinde yeni yaklaşımlar diye adlandırılabilir. Hoca burada son dönemde ortaya çıkan problem ve uygulama türlerinden bahsetmeye çalıştığını, bunların daha güncel (up to date) bir kısım olduğunu söyledi. Anlatılanların büyük kısmı yalnız NLP için geçerli değildir; her türlü veriyle, her türlü uygulamada kullanılabilecek yaklaşımlardır. Hoca bu konuların araştırmalara yeni boyutlar katmak için faydalı olabileceğini düşündüğünü belirtti.

Önceki derste iki konu işlenmişti. **Aktif öğrenmede** eğitim örneklerinin etiketleri başta belli değildir; elimizde örnekler vardır ama çok büyük kısmının etiketi yoktur. Algoritma, etiketi olmayanlardan hangilerinin etiketini isteyeceğini seçer. Amaç etiketleme maliyetini düşürmektir.

Yarı eğiticili öğrenmede ise etiketli ve etiketsiz veriler vardır, ama etiketsiz verilerin etiketini öğrenmek için yeterli bütçe yoktur. Soru şudur: etiketsiz verilerin kendilerinden faydalanabilir miyiz? Burada tahmini etiket (pseudo-label) kavramı ortaya çıkar. Elimizdeki etiketli verilerin tamamından ya da bir kısmından bir model oluşturulur, bu modelle etiketsiz verilerin etiketi tahmin edilir, sanki etiketleri varmış gibi davranılır. Tahmini etiketli veriler gerçek etiketli verilerle birleştirilerek yeni bir model eğitilir. Bu modelle yeniden tahmin yapılarak süreç iteratif biçimde de ilerleyebilir.

Kaldığımız slayt yarı eğiticili öğrenmenin bulgularıydı (ekrandan, 00:10:00):

- Tahmini etiketleri hiç kullanmamanın da, tek seferde tahmin etmenin de, iteratif tahminin de diğerlerinden iyi olduğu sonuçlar vardır.
- İteratif tahminde nerede durulacağı sonucu etkiler.
- Derin öğrenmede yarı eğiticili yöntemler için slaytta bir derleme makalesine bağlantı verilmiştir.

2. Öğrenmenin Transferi

2.1 Tanım ve Amaç

Öğrenmenin transferi (transfer learning) şu soruyu sorar: bir şey öğrendik; bunu başka bir yere, başka bir probleme aktarabilir miyiz? Bir problemi çözdükten sonra benzer başka bir problemi sıfırdan çözmek yerine önceki verileri, modelleri, parametreleri, gerekirse dönüştürerek kullanmak hedeflenir. Amaç, yeni problemi sıfırdan çözmeye göre daha az maliyetle ya da daha iyi bir çözüm bulmaktır.

Genel öğrenme yapısında elimizde bir veri kümesi vardır ve "öğrenmek", modelin parametrelerini bu veri kümesine uyumlu hale getirmektir. Öğrenmenin transferinde en çok kullanılan yapı şudur: kocaman bir veri kümesiyle bir model eğitilmiştir ve bu model araştırmacıların erişimine açık bir yerde, "rafta" durur. Bunlara önceden eğitilmiş (pre-trained) model denir. Elimize yeni bir problem geldiğinde bu hazır modeli alıp kendi problemimiz için kullanırız. Model hiç değiştirilmeden de kullanılabilir (zero-shot; hoca bunu ileride konuşacaklarını söyledi), ama genelde bir miktar ince ayar (fine-tune) yapılır, yani model elimizdeki veri kümesine uyumlu hale getirilir.

2.2 İnce Ayar Neden Ayrı Bir Ad Alır?

Hoca ince ayarın da aslında bir eğitim olduğunu, temelde herhangi bir eğitimden farkı olmadığını vurguladı. Ayrı bir ad almasının nedeni ölçek farkıdır: önceden eğitilmiş modelin eğitimi örneğin bir milyon veriyle on gün sürmüş olabilir, sizin ince ayarınız ise belki beş yüz veriyle on dakika sürer. Birine "eğitim" dersek diğerine "eğitim" demek ayıp olur gibi bir durum vardır.

Asıl fark başlangıç noktasıdır. İlk eğitimde model genelde rastgele bir başlangıçtan, sıfırdan eğitilir. İnce ayarda ise rastgele bir ilk değerle başlanmaz; kocaman veriyle eğitilmiş modelin parametrelerinden başlanır. Eğitim parametre optimizasyonu olduğuna göre, ince ayar büyük bir veri kümesinde zaten optimize edilmiş parametrelerle başlayan bir optimizasyondur.

2.3 Neler Transfer Edilebilir?

Slayttaki liste (ekrandan, 00:14:30):

- Veriler (aynen ya da değiştirilerek)
- Tüm model
- Metinler için kelime vektörleri (sabit ya da adaptif)
- Resimler için CNN filtreleri (sabit ya da adaptif)

En çok kullanılan, tüm modelin transferidir. CNN filtrelerinin transferi de model transferine çok benzer. Slayttaki şekilde bir görüntü düşük seviyeli özelliklerden (low-level feature) orta ve yüksek seviyeli özelliklere, oradan eğitilebilir sınıflandırıcıya giden bir akışla gösterilir.

Hoca alt seviye özelliklerin veri kümesinden veri kümesine çok fazla değişmeyen temel yapılar olduğunu açıkladı. İki farklı veri kümesi üzerinde aynı CNN yapısını ayrı ayrı, sıfırdan eğitirseniz, ilk katmanlarda öğrenilen özellikler sonraki katmanlarda öğrenilenlere göre birbirine daha benzer olur. İlk katmanlardakiler genelde yön ve gradyan bilgisi gibi yapılardır; katmanlar ilerledikçe özellikler sınıfa ve veri kümesine özgü hale gelir. Bu yüzden baştaki katmanlar çok kolay transfer edilir, son katmanların transferi daha zordur; zaten son katmanlarda ince ayar yapılır.

2.4 Ne Zaman İşe Yarar?

Slayta göre (ekrandan, 00:16:10) öğrenmenin transferi iki durumda işe yarar:

- Eldeki yeni problem için yeterli veri yoksa

- Eski ve yeni problem ilgili ise

Elinizde az veri varsa ve bu veriyle modeli sıfırdan eğitmeye kalkarsanız model o kadar başarılı olmaz; ama büyük veriyle eğitilmiş modeli alıp ince ayar yaptığınızda daha başarılı olabilirsiniz. İki problem birbirinden tamamen bağımsızsa transfer uygun değildir. Hoca örnek olarak resimler için elde edilen özellikleri metinlerde kullanmayı verdi: belli dönüşümlerle yapılabilse bile yapıları farklı olduğu için çok uygun olmaz. Burada iki türü birlikte kullanarak eğitilen modellerden değil, birinden alınanın ötekine aktarılmasından bahsedilmektedir.

Slayttaki grafikte yatay eksen görev ilişkisi (task relatedness), dikey eksen transfer performansıdır. "Aggressive" çizgisi daha dik, "safe" çizgisi daha yatıktır. Slayttaki açıklamaya göre "aggressive" çok fazla bilgi transferidir; benzer bir karakteristik yarı eğiticili öğrenme için de söylenebilir.

> **Ek bilgi (derste anlatılmadı):** Grafikteki yorum büyük olasılıkla şudur: görevler ilgisiz olduğunda transfer performansı düşürebilir (negatif transfer). Agresif transfer ilgili görevlerde çok kazandırır ama ilgisiz görevlerde çok kaybettirir; güvenli transfer hem kazancı hem kaybı sınırlar.

3. Çok Görevli Eğitim

3.1 Tanım

Hoca öğrenmenin transferinin herkesin bildiği bir konu olduğunu söyleyip çok görevli eğitime (multi-task learning) geçti. Slayttaki tanım (ekrandan, 00:17:45):

- Aynı anda birden fazla şeyi öğrenmek
- Çok sayıda veri kümesini aynı yapı içinde modellemek
- Hem satranç, hem dama oynayabilen tek bir sistem

Elinizde bir veri kümesi ve ona göre tanımlanmış tek bir kayıp (loss) fonksiyonu olup yalnız onu öğrenmek yerine, çok sayıda veri kümesini aynı yapı içinde modellemek söz konusudur. Slayttaki şemada transfer öğrenmede bir kaynak görevden (source task) hedef göreve (target task) tek yönlü bir ok vardır. Kaynak görev önceden eğitilmiş modelin eğitildiği görev, hedef görev elimizdeki problemdir. Çok görevli öğrenmede ise dört görev (Task 1-4) birbirine çift yönlü oklarla bağlıdır: bir sürü görev vardır ve hepsi aynı anda, eş zamanlı ya da eş zamansız öğrenilmeye çalışılır.

3.2 Görüntü İşlemeden Bir Örnek

Slayttaki örnekte (ekrandan, 00:20:00) bir sokak görüntüsü girdi olarak verilir:

```
Input Image -> Encoder -> Semantic Decoder -> Semantic Task Uncertainty -+
                       -> Instance Decoder -> Instance Task Uncertainty -+-> Σ -> Multi-Task L
                       -> Depth Decoder    -> Depth Task Uncertainty    -+
```

Görüntü bir kodlayıcıdan (encoder) geçirilir ve belli özellikler elde edilir. Sonra üç ayrı görev vardır: anlamsal bölütleme (segmentasyon), nesne (instance) düzeyinde ayırma ve derinlik kestirimi (depth prediction). Üçü için hesaplanan değerler birleştirilerek bir çok görevli kayıp oluşturulur ve geri bildirim üçüne birden verilir.

Normalde bu üç görevin her biri ayrı ayrı eğitilebilir; standart yaklaşım budur: elinizde "bu girdiye karşı şu çıktı" çiftleri vardır ve bunlardan eğitim yapılır. Çok görevli eğitimde ise aynı girdiye karşı birden fazla çıktı vardır ve hepsi aynı anda eğitilir. Fikir, bir görevde öğrenilen bir şeyin diğerine de yardımcı olabileceğidir. Görevler birbirinden bağımsız yürüyen kollara sahiptir ama kodlayıcı üçünde de ortaktır. Nesneleri belirlerken öğrenilen bir şey derinliği bulmakta, derinliği öğrenirken öğrenilen bir şey nesneleri tanımada işe yarayabilir.

3.3 NLP Decathlon

Görüntü işlemede bu görevlerin hepsinde girdi bir resimdir, çıktılar da aynı tipte, birer resimdir. Birbirine fayda sağlayan bu görevlerde çıktının bir ortaklığı vardır. NLP Decathlon, bu ortak girdi-çıktı yapısını NLP için kurma çalışmasıdır.

NLP alanında birçok görev vardır: makine çevirisi, duygu analizi, soru cevaplama, çoktan seçmeli soru cevaplama, gerektirme (entailment) problemleri gibi. Bunların girdi-çıktı biçimleri birbirinden farklıdır. Makine çevirisinde bir girdi cümlesi ve bir çıktı cümlesi vardır. Soru cevaplamada bir paragraf, bir soru ve cevabı vardır, yani üçlü bir yapı söz konusudur. Duygu analizinde girdi bir metin, çıktı bir sınıf bilgisidir. NLP Decathlon'da bütün veri kümeleri ortak bir formata kavuşturulur: soru, bağlam, cevap (question, context, answer).

Slayttaki örneklerden bazıları (ekrandan, 00:24:20):

Soru (Question)	Bağlam (Context)	Cevap (Answer)
What is a major importance of Southern California in relation to California and the US?	…Southern California is a major economic center for the state of California and the US….	major economic center
What is the translation from English to German?	Most of the planet is ocean water.	Der Großteil der Erde ist Meerwasser
What is the summary?	Harry Potter star Daniel Radcliffe gains access to a reported £320 million fortune…	Harry Potter star Daniel Radcliffe gets £320M fortune…
Hypothesis: Product and geography are what make cream skimming work. Entailment, neutral, or contradiction?	Premise: Conceptually cream skimming has two basic dimensions – product and geography.	Entailment
Is this sentence positive or negative?	A stirring, funny and finally transporting re-imagining of Beauty and the Beast and 1930s horror film.	positive
What is the translation from English to SQL?	The table has column names… Tell me what the notes are for South Australia	SELECT notes from table WHERE 'Current Slogan' = 'South Australia'

Çeviride soru "İngilizcenin Almancaya çevirisi nedir?", bağlam İngilizce cümle, cevap Almancasıdır. Özetlemede soru "özet nedir?", bağlam metin, cevap özettir. Gerektirmede hipotez ve öncül soru ve bağlama konur; cevap ikisi arasındaki ilişkidir (gerektirme mi, çelişki mi). Duygu analizinde soru "bu cümle pozitif mi negatif mi?", cevap "positive"tir.

Böylece birbirinden farklı yapılardaki veri kümeleri ve görevler tek bir biçime getirilir. Hoca bunu, günümüzde görülen istem (prompt) tabanlı sistemlere bir geçiş olarak da değerlendirebileceğimizi söyledi: her şey tek bir biçime dönüşür. Bunun faydası, bir görevi öğrenirken kazanılan yeteneklerin başka görevlerde de kullanılabilmesidir; örneğin özetleme yaparken öğrenilen bir şey gerektirme görevinde de işe yarayabilir.

4. Birbiriyle İlgisiz Veri Kümelerini Birlikte Eğitmek

4.1 Deneyin Kurgusu

Buraya kadarki örneklerde görevler birbiriyle ilgiliydi: hepsi NLP ya da hepsi görüntü işleme veri kümeleriydi. Hoca birbirinden çok alakasız veri kümelerini birlikte eğitmenin mümkün olup olmadığını sordu ve bunun için yaptığı bir denemeyi gösterdi (kod: multi_task_learning.m; hoca kodun incelenebileceğini söyledi).

Slayttaki kurgu (ekrandan, 00:30:00): 2 boyutlu, 2 sınıflı 5 veri kümesi için 10 gizli nöronlu yapay sinir ağı, eşit eğitim süresi, 3 deneme. Beş veri kümesinin her birinde iki özellik ve iki sınıf vardır, ama ne boyutları ne de sınıfları birbiriyle ilgilidir. Hoca bunları bir sentiment veri kümesi ile bir oyunun kim tarafından kazanılacağını tahmin eden, taş pozisyonları içeren bir veri kümesi kadar ilgisiz diye niteledi.

- **Single:** her veri kümesi tek gizli katmanı 10 nöronlu bir ağla ayrı ayrı eğitilir.

- **Multi:** beş veri kümesinden tek bir veri kümesi oluşturulur ve tek bir ağ eğitilir.

Birlikte eğitirken modele örneğin hangi veri kümesinden geldiğinin söylenmesi gerekir. Her veri kümesinin iki girdisi vardır; hepsini birden eğitirken üçüncü bir boyut gibi bir şeye, örneğin bir veri kümesi kimliğine (ID) ihtiyaç vardır: "bu, şu numaralı veri kümesinin bir örneğidir, çıktısını ona göre üret." Hoca bu olmadan, birbirinden her şeyleriyle bağımsız olan veriyi anlamlı biçimde kodlamanın mümkün olmadığını söyledi.

4.2 Sonuçlar

Slayttaki sonuçlar (ekrandan, 00:30:00). `tr_acc` eğitim, `acc` test doğruluğudur; sütunlar beş veri kümesidir:

```
Deneme 1
single_tr_acc = 1.0000  0.5875  0.8462  0.9714  0.7663
single_acc    = 0.9067  0.5017  0.6923  0.9771  0.7554
multi_tr_acc  = 0.9333  0.5611  0.7483  0.9657  0.7609
multi_acc     = 0.8667  0.4785  0.7343  0.9656  0.7935

Deneme 2
single_tr_acc = 1.0000  0.5479  0.8671  0.9771  0.7989
single_acc    = 0.9467  0.5050  0.7063  0.9656  0.7228
multi_tr_acc  = 0.8133  0.5446  0.7692  0.9600  0.7935
multi_acc     = 0.7200  0.4983  0.7413  0.9656  0.7446

Deneme 3
single_tr_acc = 0.9733  0.5380  0.8112  0.9686  0.7935
single_acc    = 0.9200  0.4521  0.6783  0.9656  0.7772
multi_tr_acc  = 0.9600  0.4785  0.7692  0.9571  0.7446
multi_acc     = 0.9600  0.4884  0.7203  0.9570  0.7772
```

Hoca ilk denemeyi okudu: tek başına eğitimde birinci veri kümesinde eğitim başarısı %100, test başarısı yaklaşık %90'dır. Birlikte eğitimde birinci veri kümesinde test başarısı yaklaşık 4 puan düşmüştür (0.9067 → 0.8667). Bazı veri kümelerinde ise işler o kadar kötüye gitmemiş, bazılarında biraz iyileşmiştir.

Test başarısındaki farklar (multi - single), python ile hesaplanmıştır:

Deneme	VK1	VK2	VK3	VK4	VK5
1	-0.0400	-0.0232	+0.0420	-0.0115	+0.0381
2	-0.2267	-0.0067	+0.0350	0.0000	+0.0218
3	+0.0400	+0.0363	+0.0420	-0.0086	0.0000

Üçüncü veri kümesinde test başarısı üç denemede de yaklaşık 3.5-4 puan artmıştır; birinci veri kümesinde ise ikinci denemede büyük bir düşüş (0.9467 → 0.7200) vardır. Hocanın özeti: korkunç kötü bir şey olmamıştır, iyileşmenin olabildiği yerler vardır.

Hoca deneyin asıl ilginç tarafının şu olduğunu söyledi: bir veri kümesini bir modele verip eğitebildiğimizi zaten biliyoruz, ama burada aynı model, birbirinden tamamen bağımsız birden fazla veri kümesini aynı anda tutabiliyor gibi görünüyor.

4.3 Bulgu ve Açık Sorular

Slayttaki bulgu-yorum (ekrandan, 00:35:20):

- İyileşmenin olduğu veri kümeleri var.
- Ama genel bir şeyler söylemek için çok daha fazla denemeye ihtiyaç var.
- Bir ağ kaç problemi kapsayabilir?
- Ağın karmaşıklığı artarsa ne olur?

Burada beş veri kümesi bir ağa gömülebildi; sayıyı çok daha fazla artırmak, ağın karmaşıklığını artırarak daha fazlasını sığdırmak mümkün mü? Hoca bunların cevap bekleyen ilginç sorular olduğunu söyledi.

5. Çok Görevli Eğitim ve Çok Kipli Modeller

Deneyin ardından bir öğrenci, çok görevli öğrenme ile son zamanlarda popüler olan çok kipli (multimodal) modellerin benzer şeyler mi olduğunu, yoksa farklılaştıkları mı sordu.

Soru: Çok görevli öğrenme ile multimodal modeller benzer mi, farklı mı?

Cevap: Multimodal yapılarda girdi farklı biçimlerde, farklı yapılardadır: hem görüntü hem ses olabilir, metin de işin içinde olabilir, video olabilir. Modal kelimesinden kasıt kip ya da moddur, yani farklı biçimlerdeki veridir; çok görevli anlamında değildir. Bu farklı biçimler aslında girdinin içinde farklı özellikler olarak gelir: ses bir özellik, video bir özellik olarak gelir. Onları birleştirmenin farklı yolları vardır; uç uca ekleyip (concatenate) bir şey yapmak zorunlu değildir, farklı mimariler ayrı eğitilip sonuçları da birleştirilebilir. Multimodal eğitimde görevler genelde ilişkilidir, birbirinden bağımsız görevler değildir. Anahtar nokta, girdi ve çıktı yapısının içinde birbirinden farklı kiplerin bulunmasıdır.

6. Hiperparametre Optimizasyonu

6.1 Hiperparametre Nedir, Optimizasyonu Neden Zordur?

Büyük modeller, yani çok sayıda parametresi olan modeller, büyük veri kümeleriyle eğitilir ve bu doğal olarak uzun sürer. Bu eğitimi yaparken algoritmaların kendilerinin belirleyemediği bazı parametreler vardır; bunlara hiperparametre denir. Model, parametreleri veriye bakarak kendisi ayarlar; hiperparametreler ise modelin veriye bakarak öğrenemediği, mutlaka bizden beklediği parametrelerdir.

Hiperparametrelerin seçimi kullanıcıya bırakıldığı için ve kullanıcı her zaman doğru değeri bilemediği için bir deneme yanılma sürecine girilir; bu süreç hiperparametre optimizasyonudur. Hoca deneme yanılmanın, denemeler çok uzun sürmüyorsa kötü bir yaklaşım olmadığını söyledi. Ama bir deneme bir gün sürüyorsa 10 hiperparametre konfigürasyonunu denemek 10 gün sürer ve konfigürasyon sayısı 10 ile de sınırlı olmayabilir.

Özellikle derin öğrenme mimarilerinde çok fazla sayıda hiperparametre vardır. Hoca bunların büyük kısmının (örneğin %90-95'inin) varsayılan değerlerinde kullanıldığını anlattı: sonucu etkileyebilecek 100 hiperparametre varsa 95'i için önceki araştırmacıların deneyip "fena değil" dedikleri değerler kullanılır, belki kendi probleminize özgü 5 tanesi optimize edilir. Bunun nedeni işin uzun sürmesidir.

Hoca hiperparametre optimizasyonunu şöyle tanımladı: elimizdeki optimizasyon problemini çözmek için aslında başka bir optimizasyon problemi çözüyoruz. Tek bir hiperparametre düşünürsek, hiperparametre uzayının nasıl bir şey olduğunu, elimizdeki belli noktalardan geçen yapının nasıl olduğunu anlamaya çalışırız.

6.2 Izgara ve Rastgele Arama

Slayttaki soru (ekrandan, 00:40:00): hiperparametre uzayının hangi noktalarında deneme yapalım? Yöntemler:

- Izgara (grid) arama
- Rastgele arama
- Bayesian optimization

En standart olanlar ızgara arama ve rastgele aramadır; ikisi birbirine aşağı yukarı benzer.

Izgara arama: 10 denemelik bir bütçeniz ve optimize edeceğiniz bir hiperparametreniz olsun. Hiperparametrenin belli bir aralıkta gezindiği varsayılır, aralık eşit parçalara bölünür ve bu değerlerle tarama yapılır; örneğin 0'dan 1'e 0.1 adımla. Bu tek boyutlu bir ızgara aramadır; iki hiperparametre için iki boyutlu bir ızgara düşünülebilir. Slaytın küçük resminde bu, `0:0.1:1` biçiminde yazılmıştır.

> **Ders sonrası not:** Hoca 10 denemelik bütçe için aralığı “10 eşit parçaya bölüp” 0’dan 1’e 0.1 adımla taramayı örnek verdi; `0:0.1:1` dizisi uç noktalar dahil 11 değer içerir (0, 0.1, …, 1). Tam 10 deneme için adım ya da uç noktalar buna göre seçilmelidir.

Rastgele arama: 10 denemenin tamamı rastgele yapılır; örneğin 0 ile 1 arasında 10 rastgele değer seçilir ve onlarla deneme yapılır.

6.3 Bayesian Optimizasyon

Bayesian optimizasyon ikisini biraz birleştirir gibidir. Slayttaki tanıma göre (ekrandan, 00:40:00) eldeki deneme sonuçlarına göre en fazla iyileşme sağlanabilecek değeri tahmin eder. Slayttaki grafikte yatay eksen hiperparametre değeri, dikey eksen modelin başarısıdır; içi boş siyah daireler eldeki deneme sonuçları, kırmızı daire en fazla iyileşmeyi sağlayacağı tahmin edilen değerdir.

Hoca bunu adım adım anlattı. 10 denemelik bütçeniz olsun. Önce rastgele üç değer seçilir, bu değerlerle eğitim yapılır ve modelin başarıları bulunur. Şimdi yeni bir deneme hakkımız var: bu denemeyi nereden yapmak mantıklı olur?

Elimizdeki bilgi yalnız denenen noktalardır. Slaytta noktaları birleştiren bir eğri çizilmiştir, ama bu eğriyi bilmiyoruz; böyle bir şey olabilir de olmayabilir de. Bayesian optimizasyona göre en makul hareket, aralarında düşüş görülen iki nokta arasında aramaktır. Bir noktadan sonra başarıda bir düşüş olmuştur; acaba başarı arada biraz daha yükselip sonra mı düşmektedir? Dar bir aralıkta böyle bir yükselip düşme olasılığı daha düşüktür; daha geniş aralıkta ise daha yüksektir. Yöntem bu aralıkta bir deneme yapar. Sonuç, aralıkta herhangi bir yerde çıkabilir; yeni sonuca göre yeniden tahmin yapılır ve bir sonraki deneme yeri belirlenir.

Izgara ve rastgele aramadan temel farkı budur: eldeki deneme sonuçlarına bakarak bir sonraki denemeyi nereden yapmanın daha mantıklı olduğunu kestirmeye çalışır. Oldukça sık kullanılan bir yaklaşımdır. Hoca bunun kimi zaman daha az deneme ile iyi sonuçlar elde etmeyi sağladığını, ama kesin ya da optimal bir süreç olmadığını belirtti.

> **Ek bilgi (derste anlatılmadı):** Bayesian optimizasyonda denemeler arasındaki bilinmeyen başarı eğrisi genelde bir Gauss süreci gibi olasılıksal bir modelle tahmin edilir. Bu model her nokta için hem bir ortalama tahmin hem de bir belirsizlik verir. Sonraki deneme, “edinim fonksiyonu” denen ve yüksek ortalama ile yüksek belirsizliği dengeleyen bir ölçütün en büyük olduğu noktadan seçilir.

7. Aktif Örnekleme

7.1 Aktif Öğrenmeden Farkı

Hoca bunun aktif öğrenme değil aktif örnekleme (active sampling) olduğunu vurgulayarak konuya girdi. Slayttaki maddeler (ekrandan, 00:50:00):

- Bayesian optimizasyondan esinlenelim.
- Kullanıcıdan input uzayı için yeni örnekler isteyebilir miyiz? (Aktif örnekleme)
- Aktif öğrenmede etiketsiz verilerin hangilerinin çıkışlarını kullanıcıdan isteyeceğimizi belirliyorduk. Burada örneklerin kendilerini (input ve çıkış) isteyeceğiz.

Elimizdeki bir veri kümesinde, uzayın belli bölgelerinden veri vardır, belli bölgelerinden yoktur; ya da belli özelliklere sahip veriler eğitim kümesinde vardır, belli özelliklere sahip olanlar yoktur. Aktif örneklemede model, aktif öğrenmeden de yarı eğiticili öğrenmeden de daha üst seviyede bir talepte bulunur: “Şu bölgeden yeni örnek oluştur; bir deney yap, girdisi şöyle olan bir örnek bul ve bana çıktısını söyle.” Yani hem girdiyi hem çıktıyı birlikte ister.

Slayttaki şekilde Bayesian optimizasyonla deney tasarımı gösterilir: bir başlangıç eğitim kümesiyle (initial training set) Bayesian modelin tahmini yapılır, yeni bir deney ya da simülasyon yapılır, yeni sonuç modele eklenir ve tahminin belirsizlik bandı daralır.

Hocanın verdiği duygu analizi örneği: modelin elindeki veri kümesinde belli bir kelime vardır ve bu kelimenin geçtiği yorumların bazıları pozitif, bazıları negatiftir. Örneğin eğitim kümesinde 10 örnek denk gelmiş, yarısı pozitif yarısı negatif. O kelimenin nasıl bir şey olduğunun

anlaşılabilmesi için o kelimeyi içeren yeni örneklere ihtiyaç vardır. Model şöyle der: "Şu kelime ile şu kelimenin birlikte geçtiği örnekleri bana getir; önce böyle örnekler bul, sonra etiketle, sonra bana ver."

Üç yaklaşımın karşılaştırması:

- **Aktif öğrenme:** örnekler bellidir; model "şu örneğin etiketini ver" der ve hangi örneğin etiketini isteyeceğini seçer.
- **Yarı eğiticili öğrenme:** etiketi bilinmeyen örnekler olduğu gibi kullanılır.
- **Aktif örnekleme:** hem girdiyi hem çıkışı birlikte talep eder; kullanıcıdan daha fazlasını bekler.

Hoca aktif örneklemeyi, modelin eğitileceği eğitim kümesinin oluşturulmasına doğrudan müdahale eden bir yaklaşım olarak özetledi: model kendi eğitim kümesinin oluşturulmasında söz sahibidir.

> **Hocanın vurgusu:** Aktif örnekleme, günümüzdeki birçok problem için oldukça önemli bir konudur.

7.2 Hangi Bölgeden Yeni Örnek İstenmeli? (kod active_sampling.m)

Slayttaki açıklama (ekrandan, 00:51:00):

- Model varyansının en yüksek olduğu yerler = tahminlere en az güvenebileceğimiz yerler.
- Modelin bir noktadaki varyansı, farklı eğitim kümeleri için o noktanın tahminlerinin standart sapmasıdır.
- Buralardan yeni örnek istemeliyiz.

Slaytta solda, yatay ekseni 0 ile 1 arasında olan, tek girdili bir regresyon veri kümesi görülür; yaklaşık 0.2 civarında örneklerin seyrek olduğu bir boşluk kırmızı daireyle işaretlenmiştir. Sağda farklı eğitim kümeleriyle eğitilmiş birçok modelin basamaklı tahmin eğrileri vardır; tam bu boşluğun olduğu bölgede eğriler birbirinden en çok ayrışır, yani varyans en yüksektir. Model burada boşluk olduğunu fark eder ve bu boşluğu çıktılarının varyansına bakarak tespit eder: "Bu bölgede çok örneğim yok, o yüzden iyi tahmin yapamıyorum, tahminlerim yüksek varyanslı; bana bu bölgeden yeni örnek ver."

> **Ek bilgi (derste anlatılmadı):** Farklı eğitim kümeleri büyük olasılıkla eldeki eğitim kümesinden yerine koyarak örnekleme (bootstrap) ile elde edilmiştir. Sağdaki basamaklı eğriler, büyük olasılıkla ağaç tabanlı bir regresyon modelinin tahminleridir; ancak kodun hangi modeli kullandığı derste söylenmedi.

Hoca bu noktada konu anlatımını bıraktı; bir sonraki konu olan insan-makine iş birliğinin sonraki derste tamamlanacağını söyledi.

8. Ödev Sunumları: Metin Sınıflandırmada Anlamsal Uzayların Karşılaştırılması

8.1 Ödevin Konusu

Arada verilen moladan sonra öğrenciler ödevlerini sundu. Ödevin ortak çerçevesi, sunumlardan anlaşıldığı kadarıyla şudur: üç metin veri kümesi seçilir; en az biri konu sınıflandırma, diğerleri konu sınıflandırması olmayan (örneğin duygu analizi, sahte haber, spam) görevlerdir. Her veri kümesinde metinler farklı anlamsal uzaylarda temsil edilir ve sınıflandırma başarıları karşılaştırılır:

- Klasik yöntemler: TF ve TF-IDF; birçok sunumda 2-6 karakter n-gramları ile
- Dağılımsal anlambilim yöntemleri: LSA, LDA, birlikte geçme matrisi (ve SVD)
- Sinirsel dil modelleri: word2vec, GloVe, FastText, Doc2Vec, BERT, RoBERTa, ALBERT, GPT-2, DistilBERT

Ayrıca bazı temsiller birleştirilip sonuçları karşılaştırılmıştır. Değerlendirme çoğunlukla 5 katlı çapraz geçerleme ile ortalama doğruluk ve standart sapma üzerinden yapılmıştır. Hoca sunumlarda giriş kısımlarının atlanmasını, asıl kritik olanın sonuçlar ve sonuçlara dair yorumlar olduğunu söyledi.

8.2 Temsilleri Birleştirme Yolları

Sunumlarda kullanılan birleştirme yöntemleri:

- **Uç uca ekleme (concatenation):** iki modelin vektörleri yan yana konur. Örneğin 768 boyutlu BERT ve 768 boyutlu RoBERTa vektörleri birleşince 1536 boyutlu bir vektör elde edilir. Bir sunumda 50.000×400 boyutlu iki temsil birleştirilince 50.000×800 boyutlu bir matris elde edilmiştir.
- **Yarıya bölme (two split):** BERT vektörünün ilk yarısı ile RoBERTa vektörünün ikinci yarısı alınır, yine 768 boyutlu bir vektör oluşur.
- **Çeyreklere bölme (four split):** dört sinirsel dil modelinin (BERT, GPT-2, RoBERTa, ALBERT) her birinin dörtte biri alınıp yan yana konur, 768 boyutlu bir vektör oluşur.
- **Eleman eleman toplama:** vektörler aynı boyuttaysa toplanır, boyut değişmez.
- **Ortalama alma:** vektörlerin ortalaması alınır. Bir sunumda 100 boyutlu hazır GloVe ile 100 boyuta ayarlanıp eğitilen word2vec uç uca eklenince 200 boyut, toplanınca ya da ortalaması alınınca 100 boyut elde edilmiştir.

8.3 Öne Çıkan Bulgular

Sinirsel modeller genelde daha iyi, ama her zaman değil. Birçok sunumda sinirsel dil modelleri TF ve TF-IDF'ten belirgin biçimde daha iyi çıkmıştır. Bir öğrencinin yorumu: TF ve TF-IDF yalnız frekansa göre çıkarım yapar, sinirsel dil modellerinin ise bağlamı da öğrendiklerini düşünüyoruz. Ancak bazı sunumlarda TF ya da TF-IDF en iyi sonucu vermiştir. Sunanlar bunu küçük ya da kısa metinli veri kümelerine (yalnız kısa özetler, tweetler), gündelik dile ve verinin bir kısmının kullanılmasına bağladı; hoca ise bazı durumlarda asıl sebebin BERT'e yanlış ön işleme yapılması olduğunu gösterdi (bkz. 9.1).

Film türü sınıflandırması (26 sınıf). Bir sunumda IMDB film özetlerinden 26 farklı türün tahmin edildiği veri kümesinde sonuçlar şöyledir (ekrandan, 01:30:00):

Yöntem	Ortalama (Acc)	S.Sapma
BERT	63	0.54
FASTTEXT	68	0.63
LSA	59	0.42
LDA	61	0.57
TF	53	0.48
TF/IDF	55	0.45
LSA-LDA	60	0.52
LSA-TF/IDF	57	0.56
LDA-TF/IDF	58	0.58

Aynı sunumda olumlu-olumsuz yorum veri kümelerinde (IMDB, Yelp) FastText 84 ve 86 ile en iyisi olmuş, onu BERT izlemiştir; LDA, LSI'dan iyi çıkmıştır. Öğrencinin amacı sinirsel olmayan yöntemleri birleştirerek sinirsel modellerin başarısını yakalamaktı; birleştirmeler iki yöntemin arasında değerler vermiş, sinirsel modellere yaklaşamamıştır. 26 sınıflı veri kümesindeki düşük başarı sınıf sayısının fazlalığına ve sınıf dağılımının dengesizliğine bağlanmıştır (ekrandaki grafikte drama ve belgesel sınıfları çok baskındır).

Dengesiz bir spam veri kümesi. Bir sunumda 5500 örnekli, yalnız yaklaşık %13'ü spam olan bir e-posta veri kümesinde 5 katlı çapraz geçerleme sonuçları şöyledir (ekrandan, 02:00:00):

Yöntem	Ortalama doğruluk	Standart sapma
BERT	96.98	0.42
GPT-2	98.62	0.55
Word2Vec	96.91	0.71
Co-occurrence Matrix	96.14	0.90
TF (2-6 karakter n-gram)	91.00 (200 örnekle)	4.64 (200 örnekle)
TF-IDF (2-6 karakter n-gram)	92.00 (200 örnekle)	4.85 (200 örnekle)
BERT + Word2Vec	97.51	0.33

Yöntem	Ortalama doğruluk	Standart sapma
GPT-2 + Word2Vec	98.58	0.47
BERT + GPT-2	98.53	0.51

Bu sunumda cümle temsilleri kelime temsillerinin ortalamasıyla bulunmuş, sınıflandırıcı olarak Random Forest kullanılmıştır. Birleştirmeler başarıyı düşürmemiş, BERT'in başarısını yükseltmiştir. TF'ye doküman bilgisi eklemek (TF-IDF) yaklaşık 1 puan artış sağlamıştır. Öğrenci, dengesiz veri yüzünden modellerin çoğunlukla "spam değil" dediğini, başarıların yüksek bantta yığıldığını ve bu veri kümesinde karşılaştırmanın güvenilir olmadığını söyledi. Aynı sunumda Google News ile önceden eğitilmiş word2vec, 7 sınıflı bir haber veri kümesinde BERT ve GPT-2'yi geçmiştir; öğrenci bunu word2vec'in eğitildiği verinin de haber olmasına bağladı. GPT-2 üç veri kümesinde de BERT'i geçmiştir. Genel yorum: sınıfların dengeli olduğu ve başarıların yüksek sınırlarda yığılmadığı deneyler temsiller arasındaki farkları daha anlaşılır biçimde ortaya koyar.

Karakter n-gramlarının maliyeti. 2 ile 6 arasındaki bütün karakter n-gramları alınınca özellik sayısı çok yükselir. Birkaç sunumda bu yüzden her sınıftan 100 örnek gibi küçük alt kümelerle çalışılmak zorunda kalınmıştır. Bir öğrenci yalnız 3'lü n-gramların da kullanılabileceğini belirtti. Bir başkası, pencere boyutu 2 olan birlikte geçme matrisinin çok büyük ve seyrek olduğunu, bu yüzden düşük başarı verdiğini düşündü.

Kararlılık. İstatistiksel yöntemler (TF, TF-IDF gibi) belli bir başlangıç değerine bağlı olmadığı için bir kere de on kere de eğitilse aynı sonucu verir; birkaç sunumda standart sapmaları sıfır çıkmıştır. Sinirsel modeller ise rastgele bir başlangıç değeriyle başlayıp gradyan temelli bir eğitimden geçtiği için, çalıştırmadan çalıştırmaya farklı doğruluklar verebilir.

Ortalama alma en kararlı birleştirme. Bir sunumda (tweet tabanlı veri dahil) TF ve TF-IDF'in yüksek çıkması ve word2vec ile GloVe'un onlara yakın kalması beklenmedik bulunmuştur; sunan öğrenci tweetlerdeki farklı yazım ve sözcüklerin bunda etkili olduğunu düşündü. Uç uca eklemenin hem boyut artışı hem anlam bozulması yüzünden kötü sonuç vermesi beklenmiş, ama şaşırtıcı biçimde öyle olmamıştır. En kararlı sonucu, word2vec ve GloVe'un arasında değerler veren ortalama almıştır.

Toplamada normalizasyon. Başka bir sunumda FastText ve BERT vektörlerini birleştirmenin üç yolu (uç uca ekleme ve toplama dahil) arasında büyük fark çıkmıştır; toplama kötü sonuç vermiştir. Sunan öğrenci, vektörleri toplamadan önce normalize etmesi gerektiğini ve iki modelin vektörlerinin birbiriyle uyumsuz olabileceğini söyledi. Aynı sunumda TF-IDF karakter n-gramları FastText'i geçmiştir; öğrenci bunu FastText'in çok büyük ve genel bir derlemde, TF-IDF'in ise her göreve özgü veride hesaplanmasına bağladı. LDA konu modellemede iyi çalışırken duygu analizi gibi görevlerde LSA kadar iyi olmamıştır.

BERT, LSTM, CRF ve katman dondurma. Bir sunumda BERT ve RoBERTa'nın Türkçe sürümleri üzerine iki farklı sınıflandırıcı (iki katmanlı tam bağlantılı ve çift yönlü LSTM) konmuş, her biri katman dondurmalı ve dondurmasız denenmiştir. Varlık ismi tanıma gibi kelime (token) düzeyindeki bir sınıflandırma görevinde LSTM eklemek tam bağlantılı sınıflandırıcıya göre belirgin iyileşme sağlamıştır; öğrenci bunu LSTM'in token token ve çift yönlü işlemesine bağladı. Kelimelere sözlük numarası veren bir "word index" temsili üzerine çift yönlü LSTM ve CRF katmanı konunca da beklenenden yüksek, %94 civarında bir sonuç alınmıştır. RoBERTa, BERT'ten farklı olarak token type id parametresi almaz; bu yüzden aynı ince ayar kodu iki modelde de kullanılabilmiştir. Katman dondurma analizi (ekrandan, 02:20:00):

- Katman dondurmak eğitimi oldukça hızlandırır, çünkü geri yayılımda temel model tarafında hesaplama yapılmaz.
- Temel modelin parametreleri tamamen aynı kaldığı için yalnız eğitilen katmanların parametrelerini kaydedip yüklemek yeterlidir (LSTM için yaklaşık 8 MB, tam bağlantılı sınıflandırıcı için KB seviyesinde); bu bellek avantajı sağlar.
- Dezavantajı, doğrulukta daha düşük değerler alınmasıdır.

Öğrenci dondurmanın, hangi mimarinin daha mantıklı olduğunu hızlıca denemek için uygun olduğunu, doğrulukla hız arasında bir ödünleşim (trade-off) olduğunu ve karar görevine göre verilmesi gerektiğini söyledi.

Diğer gözlemler.

- GPT-2'nin sınıflandırma görevi için eğitilmemiş olması, bir sunumda BERT'e göre düşük kalmasının olası nedeni olarak gösterildi.
- 4 sınıflı dengeli bir haber veri kümesinde hiçbir öğrenme yapılmasa (Zero Rule) doğruluk %25 olur; bir sunumda BERT, Rotten Tomatoes veri kümesinde %54 ile iki sınıflı bir veride rastgele tahmine (%50) yakın kalmıştır. Aynı sunumda BERT ile GloVe birleştirilince BERT tek başına %54, GloVe %67 verirken birleşim %70.41'e çıkmıştır; öğrenci tek başına kötü sonuç veren uzayları birleştirmenin daha iyi sonuç verebildiğini söyledi.
- Bir sunumda NLTK ile tokenizasyon, stop word temizliği ve WordNet lemmatizer kullanılmış; BERT ve GPT ile en iyi sonuçlar (birleşimde yaklaşık %97) Reuters haber veri kümesinde alınmıştır.

9. Sunumlarda Hocanın Geri Bildirimleri

9.1 BERT'e Ön İşlenmiş Metin Verilmez

Birkaç sunumda metinler stop word temizliği, sayı ve noktalama silme, lemmatizasyon gibi işlemlerden geçirildikten sonra BERT'e verilmişti. Hoca her seferinde BERT'e verirken bu işlemlerin yapılıp yapılmadığını sordu.

> **Hocanın vurgusu:** BERT'in kendi tokenizer'ı vardır; dolayısıyla ona verirken bu tarz işlemleri yapmamanız gerekiyor. Diğer yöntemlerde işe yarayabilirler, ama BERT'te işi bozarlar.

Bu hatayı yapan bir sunumda BERT beklenenden çok düşük çıkmış, TF ve TF-IDF en iyi sonucu vermişti; hoca bunun BERT'in değil yapılan ön işlemenin suçu olduğunu söyledi. Bir öğrenci de BERT'in çift yönlü olduğu, önündeki ve arkasındaki kelimelere baktığı için sayıların ve stop word'lerin çıkarılmasının anlamsal bozulmalara yol açmış olabileceğini yorumladı.

9.2 Karşılaştırma Adil Olmalı

Bir sunumda FastText kendi algoritması ve sınıflandırıcısıyla, diğer bütün temsiller ise Random Forest ile sınıflandırılmıştı ve FastText hep açık farkla öndeydi.

Soru (hoca): FastText'te ne yaptın, diğerlerinde Random Forest mı kullandın?

Cevap: FastText kendi modeliyle eğitildi; diğerleri Random Forest'a verildi.

Hoca bunun karşılaştırmada sorun yarattığını açıkladı: FastText'in devamında muhtemelen bir yapay sinir ağı kullanılıyor. Vektörler doğrudan çekilip aynı Random Forest'a verilseydi sonuçlar karşılaştırılabilir olurdu. Bu haliyle temsil manasında "bu daha iyi, bu daha kötü" denemez; diğer yöntemler kendi aralarında karşılaştırılabilir, ama FastText hariç tutulmalıdır.

Benzer biçimde hoca başka sunumlarda BERT temsilinin sınıfının nasıl bulunduğunu sordu: BERT ince ayar mı yapılıyor, yoksa temsil alınıp harici bir sınıflandırıcıya mı veriliyor? Bir sunumda modeller ince ayar yapılarak eğitilmiş, diğer yöntemlerde vektörler bir sınıflandırıcıya verilmişti.

9.3 Birleştirilen Temsiller Birbirini Tamamlamalı

Bir sunumda yalnız istatistiksel temsiller (TF-IDF ile TF-IDF'in bir çeşidi, TF-IDF ile LDA) birleştirilmişti.

Soru (hoca): Birleştirmede birbirini tamamlayıcı şeyleri birleştirmek gerekmez mi?

Hoca, en iyi sonucu veren FastText'in başarısını ne kadar artırabileceğine bakmak için, örneğin FastText'e BERT'i eklemenin daha mantıklı bir hareket olacağını söyledi. Benzer bir soru, TF'yi Hashing ve Count Vectorizer ile birleştiren bir sunuma da soruldu: başarılı bir temsili tamamlayıcı bir temsille birleştirerek daha yukarı çekmek mümkün olabilirdi.

9.4 Dengesiz Veride Doğruluk ve F1

Soru (hoca): Başarı değerlerinde doğruluk (accuracy) mı, F1 skoru mu kullandın?

Cevap: Doğruluk kullanıldı; öğrenci F1 kullansaydı dengesiz veri kümeleri için daha dengeli sonuçlar bulabileceğini kabul etti. Hoca da bunun iyi olacağını onayladı.

9.5 Kendi Eğitilen word2vec ve Sınıflandırıcı Seçimi

Bir sunumda word2vec sıfırdan eğitilmiş, metin temsili en fazla özellik sayısı 500 verilerek 500 boyutlu yapılmış ve Naive Bayes ile sınıflandırılmıştı. Hoca Naive Bayes yerine farklı algoritmalar denenip denenmediğini sordu. Öğrenci başta vektörler çok büyükken yalnız Naive Bayes'in sonuç verebildiğini, boyut küçültüldükten sonra denenen bir başka algoritmayla büyük fark görmediğini söyledi.

9.6 Çalışmayan Model

Bir sunumda GPT'nin çok düşük sonuçlar verdiğini gören hoca, GPT'nin aslında çalıştırılamamış olduğunu, kodda bir sorun bulunduğunu söyledi.

Örnek Sorular

1. (derste sözlü, 00:43:01) Hiperparametre optimizasyonunda 10 denemelik bütçe var. Rastgele seçilen üç hiperparametre değeriyle eğitim yapılıp başarıları bulunmuştur; ortadaki noktadan sağdaki noktaya geçerken başarı düşmüştür ve bu iki nokta arasındaki aralık, diğer aralıktan geniştir. Bayesian optimizasyona göre dördüncü deneme nereden yapılmalıdır?

Çözüm (derste verildi): Aralarında düşüş görülen ve daha geniş olan aralıktan yapılmalıdır. Bu aralıkta başarının önce yükselip sonra düşmüş olma, yani denenmemiş bir tepe bulunma olasılığı daha yüksektir; dar aralıkta böyle bir tepenin olma olasılığı daha düşüktür. Deneme sonucuna göre tahmin güncellenir ve sonraki deneme yeri yeniden seçilir.

2. (derste sözlü, 00:33:02) Çok görevli öğrenme ile çok kipli (multimodal) modeller arasındaki fark nedir?

Çözüm (derste verildi): Multimodal modellerde girdi farklı biçimlerdedir (görüntü, ses, metin, video); bu kipler girdinin farklı özellikleri olarak gelir ve görevler genelde ilişkilidir. Çok görevli öğrenmede asıl nokta, birden fazla görevin (çıktının) aynı yapı içinde birlikte öğrenilmesidir.

3. (ekrandan, 00:30:00) İki boyutlu, iki sınıflı ve birbiriyle ilgisiz beş veri kümesi tek bir yapay sinir ağında birlikte eğitilecektir. Ağa ek olarak ne verilmelidir ve ilk denemede birlikte eğitimin test başarısını hangi veri kümelerinde artırdığı görülmüştür?

Çözüm (derste verildi, farklar python ile hesaplandı): Ağa örneğin hangi veri kümesinden geldiği (veri kümesi kimliği gibi ek bir giriş) verilmelidir. İlk denemede test başarısı üçüncü (0.6923 → 0.7343, +0.042) ve beşinci (0.7554 → 0.7935, +0.038) veri kümelerinde artmış; birinci (0.9067 → 0.8667, -0.040), ikinci (-0.023) ve dördüncü (-0.012) veri kümelerinde azalmıştır.

4. (ekrandan, 00:51:00) Aktif örneklemede yeni örnek girdi uzayının hangi bölgesinden istenmelidir ve modelin bir noktadaki varyansı nasıl hesaplanır?

Çözüm (derste verildi): Model varyansının en yüksek olduğu bölgeden istenmelidir; bu bölgeler tahminlere en az güvenebileceğimiz yerlerdir. Modelin bir noktadaki varyansı, farklı eğitim kümeleriyle eğitilen modellerin o noktadaki tahminlerinin standart sapmasıdır.

5. (derste sözlü, 02:32:27) Metinlerde kullanıcı adları, bağlantılar, alfanümerik olmayan karakterler temizlenip lemmatizasyon yapıldıktan sonra BERT'e verilmiştir. Bu doğru mudur?

Çözüm (derste verildi): Değildir. BERT'in kendi tokenizer'ı vardır; ona verilecek metinde bu tarz ön işlemeler yapılmamalıdır. Bu işlemler diğer yöntemlerde işe yarayabilir, ama BERT'te işi bozar ve başarıyı düşürür.

6. (derste sözlü, 02:42:02) FastText kendi sınıflandırıcısıyla, diğer temsiller Random Forest ile sınıflandırılmıştır ve FastText hep en iyi çıkmıştır. Buradan "FastText en iyi temsildir" sonucu çıkarılabilir mi?

Çözüm (derste verildi): Çıkarılamaz. Sınıflandırıcılar farklı olduğu için fark temsilden değil sınıflandırıcıdan da kaynaklanabilir. FastText vektörleri çekilip aynı Random Forest'a verilmeli ya da FastText karşılaştırmanın dışında tutulmalıdır.

7. (derste sözlü, 02:05:07) Sınıfların dengesiz olduğu ve başarıların %96-98 bandında yığıldığı bir veri kümesinde temsilleri karşılaştırmak için hangi ölçüt daha uygundur?

Çözüm (derste verildi): Doğruluk (accuracy) yerine F1 skoru daha dengeli sonuç verir.

Dersin Toparlaması

Dersin ilk yarısında Bölüm 4'te makine öğrenmesinin güncel yaklaşımlarına devam edildi. Öğrenmenin transferinde önceden eğitilmiş bir modelin küçük veriyle ince ayar yapılarak kullanılması, bunun ne zaman işe yaradığı (yeni problem için yeterli veri yoksa ve problemler ilgiliyse) ve CNN'de ilk katmanların neden kolay aktarıldığı anlatıldı. Çok görevli eğitimde ortak bir kodlayıcı ve göreve özgü çözücülerle birden fazla görevin birlikte öğrenilmesi, NLP Decathlon'un görevleri soru-bağlam-cevap biçimine dönüştürmesi ve hocanın birbiriyle ilgisiz beş veri kümesini tek bir ağda eğittiği deney incelendi. Hiperparametre optimizasyonunda ızgara, rastgele ve Bayesian aramanın farkı; aktif örneklemede ise modelin, varyansının yüksek olduğu bölgelerden hem girdi hem çıkışı içeren yeni örnekler isteyerek kendi eğitim kümesinin oluşturulmasına müdahale etmesi ele alındı.

İkinci yarıda öğrenciler metin sınıflandırmada klasik, dağılımsal ve sinirsel anlamsal uzayları karşılaştırdıkları ödevlerini sundu. Sinirsel dil modelleri çoğu zaman öne çıktı, ama sonuçlar veriye, ön işlemeye ve sınıflandırıcıya çok bağlıydı. Hocanın geri bildirimleri deney tasarımına dair genel dersler içeriyordu: BERT gibi modellere ön işlenmiş metin verilmez, karşılaştırmada aynı sınıflandırıcı kullanılır, birleştirilen temsiller birbirini tamamlamalıdır ve dengesiz veride F1 tercih edilir. Hoca, Bölüm 4'te kalınan insan-makine iş birliği konusunun sonraki derste tamamlanacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.